Explanation

Brute Force: List

Time & Space Complexity Analysis

Time Complexity

Operation	Complexity	Reason
`add(key)`	**O(1)**	Direct index assignment
`remove(key)`	**O(1)**	Direct index assignment
`contains(key)`	**O(1)**	Direct index lookup

All operations are pure array index access — no loops, no searching, no hashing needed. It doesn't get faster than this.

Space Complexity

	Complexity	Reason
Overall	**O(1)**	Fixed-size array, independent of input

This might feel counterintuitive, but since the array is **always** allocated at a fixed size (`10^6 + 1`) regardless of how many keys are inserted, it's technically O(1) — constant space.

However, this is a case where Big-O hides the real cost:

```
[False] * (10**6 + 1)
≈ 1,000,001 booleans
≈ ~8 MB in memory — always, even for 1 insertion
```

The Core Tradeoff

```
Brute Force          vs        Hash-based (optimized)
──────────────────────────────────────────────────────

Time  → O(1) ✅                Time  → O(1) avg ✅
Space → O(1) but ~8MB 😬       Space → O(n) but scales with i
nput ✅
```

> The brute force trades **memory efficiency for speed** — it wins on time complexity but wastes space when the number of actual keys inserted is small relative to the key range.

This is exactly why the optimized approach (hashing + chaining) uses **O(n)** space — it only allocates what it needs based on actual insertions.

Hashing + Chaining

Time & Space Complexity Analysis

Time Complexity

Operation	Average Case	Worst Case	Reason
`add(key)`	**O(n/k)**	**O(n)**	Traverse chain at hashed bucket
`remove(key)`	**O(n/k)**	**O(n)**	Traverse chain at hashed bucket
`contains(key)`	**O(n/k)**	**O(n)**	Traverse chain at hashed bucket

Where:

- **n** = number of keys inserted
- **k** = number of buckets (`10^4`)
- **n/k** = average chain length per bucket (load factor)

Worst case O(n) occurs when all keys hash to the same bucket — every operation degrades to a linear scan of one long chain.

Space Complexity

	Complexity	Reason
Overall	**O(k + n)**	Bucket array + inserted nodes

Breaking it down:

```
k dummy nodes  →  10^4 ListNodes (always allocated)
n real nodes   →  1 ListNode per insertion (scales with inpu
t)

Total = O(k + n)  →  simplifies to O(n) since k is constant
(10^4)
```

Brute Force vs Optimized

```
                Brute Force          Optimized (Chaining)
                ──────────────────────────────────────────
Time            O(1)                 O(n/k) avg | O(n) worst
Space           O(1) but ~8MB fixed  O(n) scales with input
```

> The optimized solution trades the **guaranteed O(1) time** of brute force for **much better memory efficiency** — it only allocates nodes for actual insertions instead of pre-allocating 1M slots.

The sweet spot between both is a good hash function that **distributes keys uniformly**, keeping chains short and pushing average time toward O(1).